essential that an API take basic precautions to avoid producing output that might be interpreted as a script when processed by a web browser.

2.6.1 Exploiting XSS Attacks

To understand the XSS attack, let’s try to exploit it. Before you can do so, you need to add a special header to your response to turn off built-in protections in most browsers that will detect and prevent reflected XSS attacks. This doesn’t mean that you shouldn’t worry about reflected XSS attacks, because attackers have found ways around the browser protections before, and Microsoft recently announced its intention to remove the XSS protection from its Edge browser.[4] But for now, this protection makes it harder to pull off this specific attack, so you’ll disable it by adding the following header filter to your `Main` class (an `afterAfter` filter in Spark runs after all other filters, including exception handlers). Open the Main.java file in your editor and add the following lines to the end of the main method:

```
afterAfter((request, response) -> {
response.header("X-XSS-Protection", "0");
});
```

The `X-XSS-Protection` header is usually used to ensure browser protections are turned on, but in this case, you’ll turn them off temporarily to allow the bug to be exploited. With that done, you can create a malicious HTML file that exploits the bug. Open your text editor and create a file called evil.html and copy the contents of listing 2.10 into it. Save the file